

CSA1711 - Artificial Intelligence

CO2 Assessment Tool - 1

Scenario Based Assignment - Answer Script

Question 1: AI-Powered Drones for Flood Relief (Greedy Best-First Search vs A*)

Scenario: A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic - roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to a heuristic estimate (straight-line distance), partial real-time updates about blocked paths, and variable movement costs depending on terrain and weather. Due to urgency, the drone must quickly decide routes with minimum delay and risk.

i. Behaviour of Greedy Best-First Search

Greedy Best-First Search (GBFS) expands the node that appears closest to the goal according to the heuristic function $h(n)$, which in this case is the straight-line distance to the drop-off point. It completely ignores the actual path cost $g(n)$ already incurred. In the flood scenario, the drone would always fly toward the geographically nearest-looking waypoint without considering whether the terrain along that direction is currently flooded, has high wind resistance, or is blocked.

Strengths

- Very fast, since it explores only in the direction the heuristic favours - useful when a quick, 'good enough' decision is needed under time pressure.
- Low memory and computation overhead, which suits a resource-constrained onboard drone processor.

Weaknesses

- Not optimal - it can choose a path that looks short in straight-line distance but is actually blocked or has very high real terrain/weather cost.
- Not complete in general graphs with cycles or dead-ends, and can get trapped oscillating near obstacles (e.g., a flooded area that lies directly between the drone and the goal).
- Highly vulnerable to uncertainty: because it ignores accumulated cost and blocked-path updates, it may repeatedly commit to routes that turn out to be unsafe once real-time data arrives.

ii. Behaviour of A* Search

A* evaluates every node using $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost accumulated so far (distance, terrain difficulty, weather penalty) and $h(n)$ is the straight-line-distance heuristic to the goal. By combining both terms, A* balances 'how much has this route already cost' with 'how promising does it look from here.' In the drone scenario, A* would not blindly chase the geometrically closest-looking waypoint; it would weigh that estimate against the real cost of weather and terrain already paid, so a route that looks farther in a straight line but is faster/safer in practice can still be preferred.

Provided the heuristic is admissible (never overestimates the true remaining cost, which straight-line distance satisfies for a drone flying over terrain) and consistent, A* guarantees an optimal path in the current known state of the graph.

iii. Impact of Heuristic Accuracy

Aspect	Greedy Best-First Search	A* Search
Accurate heuristic	Finds a good path quickly, but still no cost guarantee.	Converges very fast to the optimal path; nodes expanded reduce sharply.

Poor/misleading heuristic	Can be led far off course - wasted delay and risk of choosing a blocked/expensive route.	Still returns an optimal solution as long as the heuristic remains admissible, though search takes longer (more nodes expanded).
Overestimating heuristic	Same greedy issues persist.	Optimality guarantee is lost - A* can then behave like GBFS and return a suboptimal route.

In short, heuristic accuracy directly determines search efficiency for both algorithms, but only A* has a built-in cost check that limits the damage of an imperfect heuristic; GBFS has no such safeguard.

iv. Effect of Dynamic Changes (Blocked Paths)

- Both algorithms are normally designed for static, fully-known environments; a sudden blocked road invalidates part of the search tree already built.
- GBFS, having committed to the 'closest looking' route, has to backtrack sharply when it discovers a block, often re-deciding from scratch with no memory of accumulated cost - this wastes critical time in an emergency.
- A* can be adapted more gracefully using incremental/real-time variants such as D* Lite or Lifelong Planning A*, which repair only the affected part of the search tree instead of restarting, since it already tracks true path cost $g(n)$.
- In both cases, frequent replanning increases computational load, so partial observability forces a trade-off between speed of response and freshness of the map.

v. Recommendation

Recommended algorithm: A* Search (ideally an online/real-time variant such as D* Lite)

- **Optimality & Safety:** A* guarantees the least-cost (and hence typically safest, lowest-risk) route given current information, which matters when delivering emergency medicine.
- **Real-time constraints:** While pure A* can be slower than GBFS, incremental real-time versions (D*, D* Lite, LPA*) preserve near-optimality while efficiently repairing the path when a road is suddenly blocked, giving a good balance of speed and correctness.
- Greedy Best-First Search may be used only as a fast fallback heuristic when computational resources are critically limited and an approximate route is acceptable, but it should not be the primary planner given the safety-critical nature of medicine delivery.

Question 2: Smart City Traffic Signal Optimization (Local Search Methods)

Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections to minimize total waiting time, fuel consumption, and traffic congestion. The system must continuously adjust signals based on traffic flow. The search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Problem Formulation as an Optimization Problem

- **State:** a complete assignment of signal timing parameters (green duration, offset, cycle length) to every intersection in the city.
- **Objective function:** a weighted combination of total vehicle waiting time, fuel consumption, and congestion level - to be minimized (equivalently, maximize a 'traffic efficiency' score).
- **Search space:** the set of all possible timing-parameter combinations across hundreds of intersections - this space grows combinatorially, making it far too large to enumerate.

Why traditional (systematic) search is inefficient: classical uninformed/informed search techniques such as BFS, DFS, or even A* are built to find an explicit path to a goal state in a state-space graph. Here there is no single 'goal state' to reach - we need the best possible configuration among an astronomically large, continuous-valued, constantly shifting space of timings. Maintaining an explicit search tree/frontier over such a huge, dynamic space is computationally infeasible and the path taken

to reach a configuration is irrelevant - only the quality of the final configuration matters. This is why the problem is better treated as a local-search / optimization problem rather than a path-finding problem.

i. Hill Climbing and its Limitations

Hill Climbing starts with an initial signal-timing configuration and repeatedly moves to a neighbouring configuration (e.g., adjusting one intersection's green time slightly) if it improves the objective function, stopping when no neighbour improves it further.

- Limitation 1 - Local maxima: it can settle on a configuration that is better than all its immediate neighbours but far worse than the true city-wide optimum.
- Limitation 2 - No backtracking or memory: once it moves, it cannot reconsider earlier states, so it cannot escape a poor region of the search space.
- Limitation 3 - Sensitivity to the starting point and to constantly changing traffic data, which keeps shifting the 'landscape' the algorithm is climbing.

ii. Local Maxima, Plateaus, and Ridges - Real-World Interpretation

Concept	Real-world traffic interpretation
Local maxima	A signal-timing pattern that looks optimal for a particular junction cluster, but a completely different citywide timing scheme would reduce congestion far more - the system gets 'stuck' improving only its local neighbourhood.
Plateaus	Many nearby timing adjustments cause almost no change in waiting time (e.g., adjusting green time by 1-2 seconds when traffic volume is flat) - the algorithm cannot tell which direction to move and effectively stalls.
Ridges	Improvement requires several intersections' timings to change together in a coordinated, sequential way (e.g., 'green wave' coordination along a corridor) - simple single-step moves miss this and zig-zag inefficiently.

iii. Overcoming the Limitations - Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA): occasionally accepts a worse configuration with a probability that decreases over time (controlled by a 'temperature' schedule). This lets the search escape local maxima and plateaus early on, while converging toward strong solutions as the temperature cools - well suited to escaping the local optima created by isolated intersection tweaks.

Genetic Algorithms (GA): maintain a population of candidate citywide timing plans, combine (crossover) good partial solutions from different plans, and introduce random mutations. This parallel, population-based exploration naturally handles ridges (by recombining coordinated timing patterns from different individuals) and avoids being trapped in one local region, and adapts well as traffic patterns change by re-evaluating fitness continuously.

iv. Recommended Approach

Recommendation: a Genetic Algorithm (optionally hybridized with local Hill Climbing for fine-tuning, and/or Simulated Annealing for single-intersection refinement)

- Scalability: GA's population-based search parallelizes naturally across hundreds of intersections and scales better with problem size than a single-point search like Hill Climbing or SA.
- Adaptability: because traffic patterns shift continuously, a population of candidate solutions can be re-evaluated and evolved online, allowing the system to track a moving optimum rather than freezing on one snapshot of traffic.
- A common practical strategy is a hybrid/memetic algorithm: GA for broad, global exploration of the huge search space, with Hill Climbing or SA used to locally polish the best candidates - combining global escape from local optima with fast local refinement.

Question 3: Autonomous Mars Rover (Online Search Agents)

Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability, and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

i. Why an Online Search Agent is Required

Offline search algorithms (like standard A* or BFS) assume the entire state space/map is known in advance, compute a full solution path before execution begins, and then simply execute it. On Mars, the rover has no prior full map of craters, rocks, or terrain hazards - the environment must be discovered through actual movement and sensing. An online search agent interleaves computation and action: it senses the local environment, decides the next best action, executes it, observes the actual outcome, and only then plans the next step. This is essential here because the map is unknown, sensing is limited, and travelling itself is the only way to gather more information about the world.

ii. Challenges of Partial Observability and Dynamic Environments

- Partial observability: the rover can only sense hazards within range of its instruments, so it may commit to a move without knowing what lies immediately beyond its sensor horizon, risking encountering an unseen crater or rock.
- Irreversible actions: some moves (e.g., driving into a loose-soil area) may be costly or impossible to undo, so mistakes from incomplete information can be expensive or mission-ending.
- Dynamic/changing terrain: dust storms, shifting sand, or lighting changes can alter previously sensed terrain, so old information can become stale or unreliable.
- Communication delay: Earth-based oversight is delayed by several minutes, so the rover largely needs to make local decisions autonomously in real time rather than waiting for remote guidance.

iii. Building and Updating the Internal Model

The rover maintains an internal map (often an occupancy grid or graph of visited/sensed cells) that starts largely unknown. As it senses its surroundings, it marks cells as free, occupied (hazard), or still unknown. Each time it moves, it merges new sensor data into this internal model, refining estimates of terrain cost and hazard likelihood. Techniques such as those used in LRTA* (Learning Real-Time A*) let the rover update the heuristic estimate of each visited state based on experience, so that if a path turns out to be more costly than expected, that information is 'learned' and used to make better decisions on future visits to the same or similar area.

iv. Online Search vs Uninformed and Informed Search

Aspect	Uninformed Search (e.g., BFS/DFS)	Informed Search (e.g., A*)	Online Search (e.g., LRTA*)
Map knowledge required	Full map needed beforehand	Full map + heuristic needed beforehand	No prior map - built incrementally through exploration
Planning vs acting	Plan fully, then act	Plan fully, then act	Interleaves planning and acting step-by-step
Handles unknown/dynamic terrain	No	No	Yes - designed for it
Computational pattern	One-time, offline	One-time, offline	Continuous, real-time, repeated

v. Justification of the Most Suitable Approach

An online, learning-based real-time search approach (such as LRTA*, using an admissible heuristic like straight-line distance to the target sample site, updated as the rover explores) is the most suitable choice. It directly matches the rover's constraints: it can operate under uncertainty because it commits to only the next action based on current knowledge, it adapts as terrain information is updated after each move, and it improves safety by refining hazard/cost estimates over repeated encounters with similar terrain rather than trusting a single stale plan computed with incomplete information.

Question 4: University Exam Timetabling (Constraint Satisfaction Problem)

Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy: no student has overlapping exams; a limited number of exam halls; certain subjects must be scheduled before others; faculty availability constraints; and special accommodations for some students. Complexity increases as the number of courses and students grows.

CSP Formulation

Variables

- One variable per course/subject that needs an exam, e.g., $X_1, X_2, \dots, X_n$ (each representing 'the exam slot & hall assigned to that course').

Domains

- For each variable X_i , the domain is the set of all valid (time-slot, exam-hall) pairs available during the exam period, e.g., $\{(Day1-9AM, Hall1), (Day1-9AM, Hall2), (Day1-2PM, Hall1), \dots\}$.

Constraints

- Unary constraint: a course requiring special accommodation (e.g., extra time, ground-floor hall) must be assigned only slots/halls satisfying that requirement.
- Binary constraint (no overlap for students): if two courses share at least one common student, their assigned time slots must differ ($X_i.slot \neq X_j.slot$).
- Resource/capacity constraint: the number of students in a course must not exceed the seating capacity of the assigned hall, and only one course may occupy a given hall in a given slot.
- Precedence constraint: if subject A must be scheduled before subject B (e.g., a prerequisite lab exam before its theory exam), then $X_a.slot$ must occur earlier in the timetable than $X_b.slot$.
- Faculty availability constraint: the invigilating faculty for a course must be free (not assigned elsewhere) during that course's slot.

ii. How Backtracking Search Works Here

Backtracking search assigns a value (a slot-hall pair) to one course variable at a time, checking after each assignment whether all constraints involving already-assigned variables are still satisfied. If a partial assignment violates a constraint (e.g., two courses sharing students end up in the same slot), the algorithm backtracks - undoes the most recent assignment(s) and tries an alternative value. This continues, using strategies like Minimum Remaining Values (choosing the most constrained course next, e.g., one with the most shared students) and Least Constraining Value (picking the slot/hall that rules out the fewest options for other courses), until a complete, constraint-satisfying timetable is found or the search proves no solution exists with the current domains.

iii. Constraint Propagation (Forward Checking)

Forward checking is applied immediately after each variable assignment: for every unassigned variable connected by a constraint to the just-assigned one, the algorithm removes from its domain any value that would now violate the constraint. For example, once Course A is fixed to Day1-9AM-Hall1, forward checking removes Day1-9AM-Hall1 from the domains of every other course, and removes Day1-9AM entirely from the domains of all courses sharing students with Course A. This prunes the search space early, detects dead-ends (a variable left with an empty domain) far sooner than plain backtracking would, and substantially reduces the number of wasted assignment attempts - critical when the number of courses and students is large.

iv. Real-World Challenges and Best Strategy for Scalability

- Challenge: with hundreds of courses and students, the constraint graph becomes extremely dense (many courses share students), so pure backtracking without propagation becomes very slow.

- Challenge: limited halls and faculty create tight resource constraints that can make some sub-problems over-constrained, occasionally requiring constraint relaxation (e.g., adding extra days) rather than a strict CSP solution.
- Challenge: special accommodations and precedence constraints add heterogeneous constraint types that must all be checked together, increasing implementation complexity.

Best strategy: combine backtracking search with forward checking (or the stronger AC-3 arc-consistency algorithm) and heuristics such as Minimum Remaining Values and Degree Heuristic (prioritising courses involved in the most constraints) for variable ordering, plus Least Constraining Value for value ordering. This combination scales far better than plain backtracking because it prunes the search space early, detects failure quickly, and focuses effort on the hardest-to-place courses first, making it practical for large, real-world university timetabling.

Question 5: Strategic Game AI (Minimax and Alpha-Beta Pruning)

***Scenario:** A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, make decisions within strict time limits, evaluate complex game states with multiple possible moves, and the decision tree is extremely large, making computation expensive.*

i. How Minimax Models the Problem

Minimax treats the game as a tree of alternating turns between two players: the MAX player (the AI, trying to maximize its evaluation score) and the MIN player (the opponent, assumed to play optimally to minimize the AI's score). The algorithm recursively explores the tree down to some depth or terminal states, assigns each leaf a value using an evaluation function, and then backs up values: MAX nodes choose the child with the highest value, MIN nodes choose the child with the lowest value. The AI ultimately selects the move at the root that leads toward the best guaranteed outcome, assuming the human opponent will always respond with their best possible counter-move.

ii. Computational Challenges of Large Game Trees

- Exponential growth: the number of nodes grows as b^d , where b is the branching factor (number of legal moves per turn) and d is the search depth - in a real-time strategy game with many possible unit actions, b can be very large, making full-depth search infeasible.
- Strict time limits: real-time gameplay demands a decision within milliseconds to a few seconds, while a full minimax search to terminal states could take far longer than is available.
- Memory constraints: storing or even traversing such large trees strains memory, especially on consumer hardware running the game client.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning adds two bounds while traversing the minimax tree: alpha (the best value MAX can guarantee so far) and beta (the best value MIN can guarantee so far). While exploring a node's children, if a value is found such that $\alpha \geq \beta$, the remaining children of that node cannot possibly affect the final decision at an ancestor node, so they are skipped ('pruned') without being evaluated. This works because once MIN has found a response good enough to make a branch worse than an alternative MAX already has elsewhere, MAX will never choose that branch, so further exploring it is wasted effort.

With a good move-ordering (evaluating the most promising moves first), Alpha-Beta pruning can reduce the effective branching factor from b to roughly the square root of b in the best case, meaning the search can go about twice as deep within the same time budget as plain minimax - directly addressing the real-time strategy game's need to evaluate more meaningful moves within strict time limits.

iv. Designing an Evaluation Function

Since the tree cannot be searched to terminal states in real time, a heuristic evaluation function estimates how favourable a non-terminal game state is for the AI. Reasonable factors to include:

- Material/resource balance: number and strength of units/resources the AI controls versus the opponent's.
- Positional advantage: territory controlled, map vision, or strategic points held (e.g., choke points, resource nodes).
- Economic strength: resource-gathering rate or production capacity, which predicts future strength beyond the current snapshot.

- Threat/vulnerability level: exposure of the AI's units or base to imminent attack, and similarly for the opponent.
- Tempo/mobility: number of available productive actions, reflecting flexibility of response.

These factors are typically combined into a weighted sum, with weights tuned (manually or via machine learning/self-play) to reflect their relative importance to winning the game.

v. Strategy for Balancing Optimality and Real-Time Performance

- Iterative deepening: run minimax with alpha-beta at increasing depth limits, always keeping the best move found so far, so a good decision is available whenever the time limit is reached, even if full-depth search isn't completed.
- Good move ordering (e.g., trying previously best moves, or captures/attacks first) to maximize the benefit of alpha-beta pruning.
- Transposition tables to cache and reuse evaluations of previously seen states, avoiding repeated work.
- A well-tuned, fast-to-compute heuristic evaluation function to substitute for full-depth search.
- Accept bounded/approximate optimality rather than perfect play - a fast, good-enough decision each turn is more valuable in real-time strategy games than a slow, provably-optimal one.